

✓ Sentiment Analysis Final Project

Project Overview

Welcome to your Sentiment Analysis final project! In this project, you will:

- 🔍 **Analyze** customer reviews or social media text data
 - ✂️ **Preprocess** text data using NLP techniques
 - 🤖 **Build** and compare multiple classification models
 - 📊 **Evaluate** model performance using appropriate metrics
 - 🗂️ **Communicate** findings in a business context
-

Project Requirements

- **Dataset:** Use a public sentiment dataset (e.g., IMDb reviews, Amazon reviews, airline tweets)
 - **Models:** Train at least **2 different classification models**
 - **Evaluation:** Use multiple metrics (accuracy, precision, recall, confusion matrix)
 - **Documentation:** Use markdown cells to explain your approach, findings, and insights
-

Important Tips for Success

Working with Limited Resources

- ⚡ **Start small:** Use `df.sample(n=1000)` or `df.head(5000)` to work with a subset
- 🎯 **Limit vocabulary:** Use `max_features=5000` in `TfidfVectorizer`
- 🚀 **Choose efficient models:** LogisticRegression and MultinomialNB are fast and effective
- 💾 **Save your work frequently:** Use Ctrl+S or Cmd+S often

Best Practices

- 📝 **Document everything:** Explain your choices and observations in markdown cells
 - 🔄 **Iterate:** Start simple, then improve
 - 📊 **Visualize:** Use plots to understand your data and results
 - 🎓 **Think like a data scientist:** Always interpret your results in context
-

Let's get started! 🚀

Double-click (or enter) to edit

✓ Part 1: Project Definition

Objectives

- Define the business problem you're solving

- Describe your chosen dataset
- Explain why sentiment analysis is valuable for this use case

Instructions

In the markdown cell below, answer these questions:

1. What is the business problem?

- What decision or insight will this sentiment analysis support?
- This sentiment analysis project aims to understand customer satisfaction with Amazon food products and to build a model that predicts review sentiment (positive, neutral, negative). This will support business decisions such as identifying product quality issues, improving customer experience, and guiding inventory or marketing strategies. The insights are valuable to Amazon sellers, brand managers, and customer service teams.
- Who would use these results?
- The goal of this project is to understand how satisfied customers are with a specific category of Amazon food products and to build a model that can automatically predict whether a review is positive, neutral, or negative. This sentiment analysis supports business decisions such as identifying product quality issues, improving customer experience, and guiding inventory or marketing strategies. The insights would be valuable to Amazon sellers, brand managers, and customer service teams who need to monitor customer satisfaction at scale.

2. What dataset are you using?

- Name and source of the dataset Amazon Fine Food Reviews - <https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews?resource=download>
- Number of samples: 568,454
- What the text represents (reviews, tweets, comments, etc.)
- This dataset consists of reviews of fine foods from amazon. The data span a period of more than 10 years, including all ~500,000 reviews up to October 2012. Reviews include product and user information, ratings, and a plain text review. It also includes reviews from all other Amazon categories.
- What are the sentiment labels (positive/negative, star ratings, etc.)?
- The sentiment labels for the Amazon Fine Food Reviews dataset are based on star ratings.

3. Why is this problem important? How could the results be used in real-world scenarios?

- This sentiment analysis project is crucial because it transforms raw customer feedback into actionable insights. In real-world scenarios, these results can be used by:
- **Product Development Teams:** To identify specific product features that receive negative feedback, guiding improvements or new product development.
- **Marketing Departments:** To understand what aspects of a product customers love, informing marketing campaigns and messaging.
- **Customer Service:** To quickly identify dissatisfied customers or emerging issues, allowing for proactive outreach and faster resolution.

- **Competitor Analysis:** To compare sentiment towards one's own products versus competitors', identifying market opportunities and threats.
 - **Inventory Management:** High positive sentiment could indicate increased demand, helping to optimize stock levels.
 - **Brand Reputation Management:** Monitoring sentiment trends can help manage and protect brand image online.
-



Your Project Definition

Business Problem:

*What decision or insight will this sentiment analysis support?

This sentiment analysis project aims to understand customer satisfaction with Amazon food products and to build a model that predicts review sentiment (positive, neutral, negative). This will support business decisions such as identifying product quality issues, improving customer experience, and guiding inventory or marketing strategies. The insights are valuable to Amazon sellers, brand managers, and customer service teams.

Who would use these results?

The goal of this project is to understand how satisfied customers are with a specific category of Amazon food products and to build a model that can automatically predict whether a review is positive, neutral, or negative. This sentiment analysis supports business decisions such as identifying product quality issues, improving customer experience, and guiding inventory or marketing strategies. The insights would be valuable to Amazon sellers, brand managers, and customer service teams who need to monitor customer satisfaction at scale.*

Dataset Description:

- **Dataset name:** Amazon Fine Food Reviews -
 - **Source:** <https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews?resource=download>
 - **Number of samples:** 568,454
 - **Text type:** This dataset consists of reviews of fine foods from Amazon. The data span a period of more than 10 years, including all ~500,000 reviews up to October 2012. Reviews include product and user information, ratings, and a plain text review. It also includes reviews from all other Amazon categories.
 - **Sentiment labels:** The sentiment labels for the Amazon Fine Food Reviews dataset are based on star ratings.
-

Importance and Real-World Applications:

This sentiment analysis project is crucial because it transforms raw customer feedback into actionable insights. In real-world scenarios, these results can be used by:

Product Development Teams: To identify specific product features that receive negative feedback, guiding improvements or new product development.

Marketing Departments: To understand what aspects of a product customers love, informing marketing campaigns and messaging.

Customer Service: To quickly identify dissatisfied customers or emerging issues, allowing for proactive outreach and faster resolution.

Competitor Analysis: To compare sentiment towards one's own products versus competitors', identifying market opportunities and threats.

Inventory Management: High positive sentiment could indicate increased demand, helping to optimize stock levels.

Brand Reputation Management: Monitoring sentiment trends can help manage and protect brand image online.

✓ Part 2: Exploratory Data Analysis (EDA) 🔍

Objectives

- Load and examine your dataset
- Understand the distribution of sentiments
- Analyze text characteristics (length, common words, etc.)
- Identify any data quality issues

What to Explore

- ✓ **Dataset structure:** Shape, columns, data types
- ✓ **Missing values:** Check for and handle missing data
- ✓ **Class distribution:** Are sentiments balanced?
- ✓ **Text length:** Average, min, max review lengths
- ✓ **Common words:** Most frequent words per sentiment
- ✓ **Sample reviews:** Display examples from each class

💡 Tips

- Use `.info()`, `.describe()`, and `.value_counts()` for quick insights
- Visualize distributions with bar plots and histograms
- Look for imbalanced classes that might affect model performance
- Create a word cloud to visualize common terms (optional but impressive!)

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
import warnings
warnings.filterwarnings('ignore')

# Set visualization style
sns.set_style('whitegrid')
```

```
plt.rcParams['figure.figsize'] = (10, 6)
```

```
print("✅ Libraries imported successfully!")
```

```
✅ Libraries imported successfully!
```

```
import os
import pandas as pd

# Load your dataset
# Assuming 'path' variable is available from previous cell execution (BC8GIJWjZksT)
# which stores the directory where the dataset files were downloaded.

# Construct the full path to the CSV file
# csv_file_path = os.path.join(path, 'Reviews.csv')
df = pd.read_csv('Reviews.csv')

pd.read_csv('Reviews.csv', nrows=10000)

# Display basic information
print(f"Dataset shape: {df.shape}")
print(f"\nColumn names: {df.columns.tolist()}")
df.head()
```

Dataset shape: (568454, 10)

Column names: ['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator', 'HelpfulnessDenominator']

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	1
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	0
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1	1
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3	3
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0	0

```
# Check for missing values
# YOUR CODE HERE
df.isnull().sum()
```

	0
Id	0
ProductId	0
UserId	0
ProfileName	0
HelpfulnessNumerator	0
HelpfulnessDenominator	0
Score	0
Time	0
Summary	1
Text	0

dtype: int64

df.info()

```
<class 'pandas.core.frame.DataFrame'>
Index: 5000 entries, 165256 to 266058
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                    5000 non-null   int64
1   ProductId             5000 non-null   object
2   UserId                5000 non-null   object
3   ProfileName           5000 non-null   object
4   HelpfulnessNumerator  5000 non-null   int64
5   HelpfulnessDenominator 5000 non-null   int64
6   Score                 5000 non-null   int64
7   Time                 5000 non-null   int64
8   Summary               4999 non-null   object
9   Text                  5000 non-null   object
dtypes: int64(5), object(5)
memory usage: 429.7+ KB
```

🌟 IMPORTANT: If working with limited resources, sample your data here
Uncomment and modify as needed:

```
df_sample = df.sample(n=5000, random_state=42)
print(f"Working with {len(df_sample)} samples")
df = df_sample # Use the sample for the rest of the project
```

Working with 5000 samples

Double-click (or enter) to edit

```
df['sentiment_count']=df['Score'].apply(lambda x: 'positive' if x>3 else ('negative' if x<3 else
```

```
# Analyze sentiment distribution

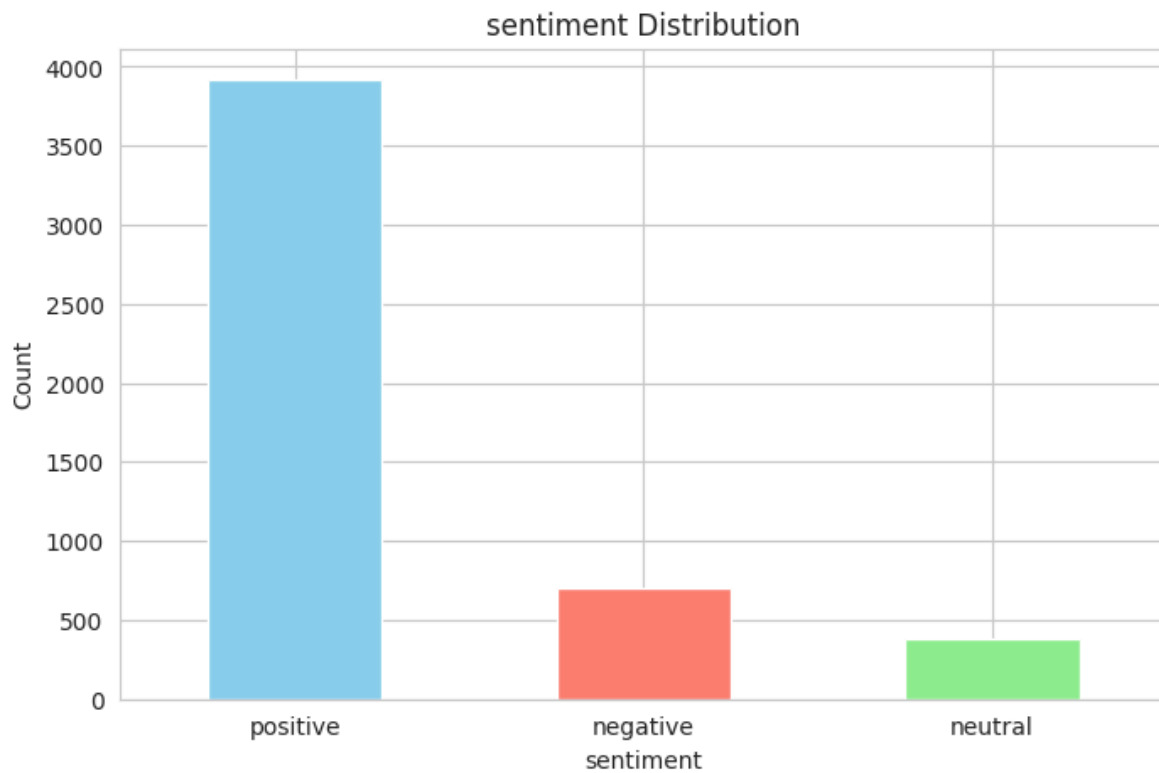
sentiment_counts = df['sentiment_count'].value_counts()

# Create a bar plot to visualize the distribution
```

```
plt.figure(figsize=(8, 5))
sentiment_counts.plot(kind='bar', color=['skyblue', 'salmon', 'lightgreen'])

plt.title('sentiment Distribution')
plt.xlabel('sentiment')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.show()

df['sentiment_count'].value_counts()
```



count	
sentiment_count	
positive	3919
negative	703
neutral	378

dtype: int64

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("snap/amazon-fine-food-reviews")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'amazon-fine-food-reviews' dataset.
Path to dataset files: /kaggle/input/amazon-fine-food-reviews

✓ Interpretation: Class Balance

Write your observations here:

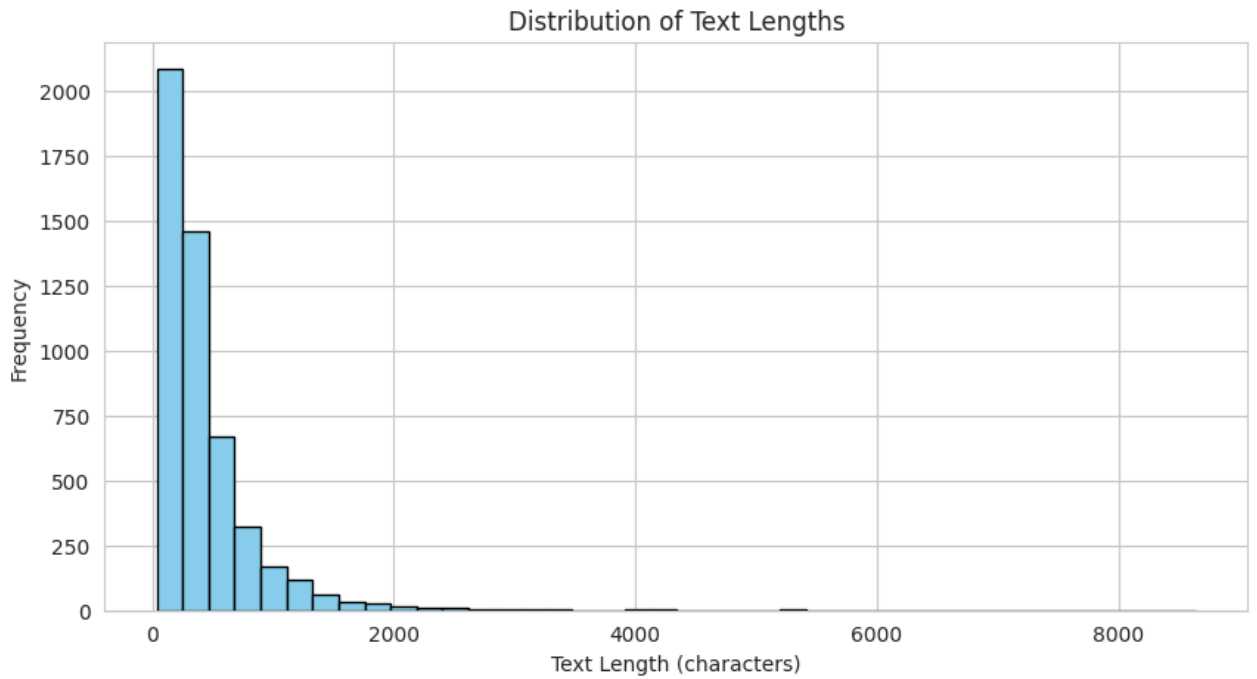
- Are the classes balanced or imbalanced?
- If imbalanced, how might this affect your model?
- What could you do to address imbalance?

```
# Analyze text length distribution

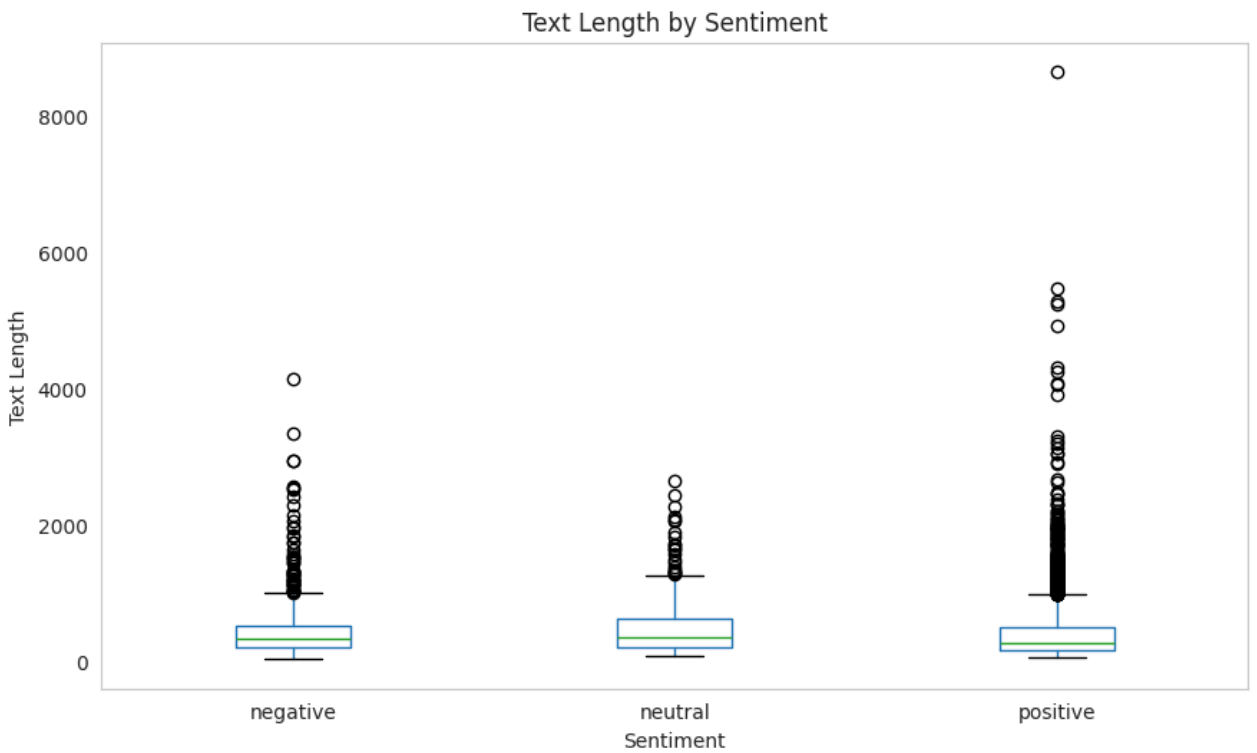
# 1. Create a new column for text length
df['text_length'] = df['Text'].str.len()

# 2. Plot histogram of text lengths
plt.figure(figsize=(10, 5))
plt.hist(df['text_length'], bins=40, color='skyblue', edgecolor='black')
plt.title('Distribution of Text Lengths')
plt.xlabel('Text Length (characters)')
plt.ylabel('Frequency')
plt.show()

# 3. Compare text lengths across different sentiments
plt.figure(figsize=(10, 5))
df.boxplot(column='text_length', by='sentiment_count', grid=False)
plt.title('Text Length by Sentiment')
plt.suptitle('') # Removes automatic Pandas title
plt.xlabel('Sentiment')
plt.ylabel('Text Length')
plt.show()
```



<Figure size 1000x500 with 0 Axes>



✓ Interpretation: Text Length

Write your observations here:

- What's the average text length?
- Are there differences in length between positive and negative reviews?
- Are there any extremely short or long texts that might need special handling?

```
# Display sample reviews from each sentiment class
# YOUR CODE HERE

# Hint: Use
df[df['sentiment'] == 'positive'].sample(3)
# Display examples from each class to get a feel for the data
```

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator
323511	323512	B003NUYBV8	A2OEUROGZDTXUJ	J. Graves	3	3
182234	182235	B000CR1VVI	A1E9EPU679WPMC	J L	0	0
351000	351001	B001EQ57DO	A10044ECXDUVKS	Dean J Copely	0	0

```
# Analyze common words (optional but recommended)
# YOUR CODE HERE

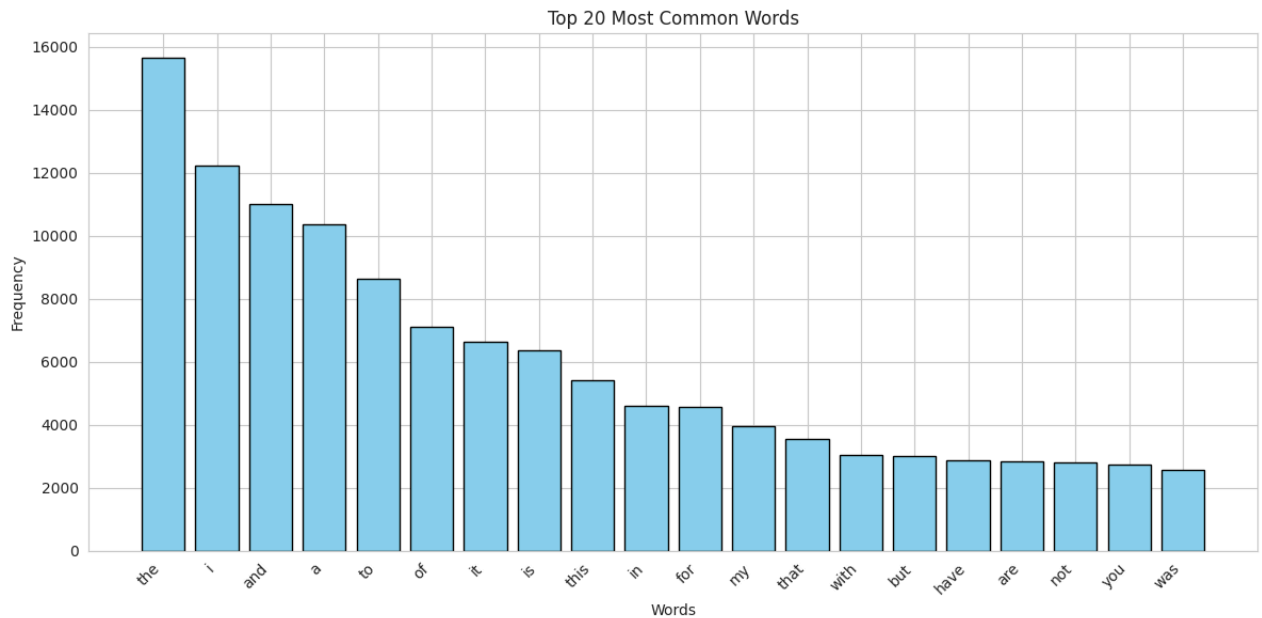
# Hint: You can do simple word frequency analysis here
# Or wait until after preprocessing for more meaningful results
# Example:
all_words = ' '.join(df['Text']).lower().split()
common_words = Counter(all_words).most_common(20)

# Separate words and counts for plotting
Words, counts = zip(*common_words)

# Plot a bar chart of most common words
plt.figure(figsize=(12, 6))
plt.bar(Words, counts, color='skyblue', edgecolor='black')

plt.title('Top 20 Most Common Words')
plt.xlabel('Words')
plt.ylabel('Frequency')
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```



EDA Summary

Summarize your key findings from the EDA:

1. **Dataset characteristics:**
2. **Data quality issues:**
3. **Key patterns observed:**
4. **Potential challenges:**

✓ Part 3: Data Preprocessing & Feature Extraction

Objectives

- Clean and preprocess text data
- Remove noise (punctuation, special characters, stopwords)
- Convert text to numerical features using TF-IDF
- Prepare data for modeling

Preprocessing Steps to Consider

- ✓ **Lowercase conversion:** Standardize text
- ✓ **Remove punctuation:** Clean special characters
- ✓ **Remove stopwords:** Filter out common words ("the", "is", "and", etc.)
- ✓ **Remove numbers:** Unless relevant to sentiment
- ✓ **Handle negations:** Be careful! "not good" vs "good" (advanced, optional)

💡 Tips

- **Don't over-preprocess:** Sometimes simple is better
- **Use `max_features` in `TfidfVectorizer`:** Limit to top 5000-10000 features to save memory
- **Consider n-grams:** Bigrams can capture phrases like "not good"
- **Test different approaches:** Try with and without certain preprocessing steps

```
# Import preprocessing libraries
import re
import string
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# For stopwords
# Option 1: Use sklearn's built-in stopwords
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

# Option 2: Use NLTK (uncomment if you prefer)
# import nltk
# nltk.download('stopwords')
# from nltk.corpus import stopwords
# stop_words = set(stopwords.words('english'))

print("✅ Preprocessing libraries imported!")
```

✅ Preprocessing libraries imported!

```
# Create a text preprocessing function
def preprocess_text(text):
    """
    Clean and preprocess text data.

    Args:
        text (str): Raw text string

    Returns:
        str: Cleaned text string
    """
    # YOUR CODE HERE

    # Hint: Steps to implement:
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'@\w+|\#\w+', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))
    text = re.sub(r'\d+', '', text)
    text = ' '.join(text.split())

    return text

# Test your function on a sample text
sample_text = "This is a TEST!!! Check out https://example.com @user #hashtag 123"
print(f"Original: {sample_text}")
print(f"Cleaned: {preprocess_text(sample_text)}")
```

Original: This is a TEST!!! Check out <https://example.com> @user #hashtag 123
Cleaned: this is a test check out

```
# Apply preprocessing to your dataset
# YOUR CODE HERE

# Hint:
df['cleaned_text'] = df['Text'].apply(preprocess_text)

# Display some examples to verify the cleaning worked

df[['Text', 'cleaned_text']].head()
print("✅ Text preprocessing complete!")
```

✅ Text preprocessing complete!

```
# Prepare your features (X) and target (y)
# YOUR CODE HERE

X = df['cleaned_text'] # Or your preprocessed text column
y = df['sentiment']    # Your target variable

# Check the shape
print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
```

Features shape: (5000,)
Target shape: (5000,)

```
# Split data into training and testing sets
# YOUR CODE HERE

# Hint:
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 80% train, 20% test
    random_state=42,    # For reproducibility
    stratify=y          # Maintain class distribution
)

print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")
```

Training samples: 4000
Testing samples: 1000

```
# Create TF-IDF features
# YOUR CODE HERE

# Hint: Initialize TfidfVectorizer with appropriate parameters
tfidf = TfidfVectorizer(
    max_features=5000,      # 🚫 IMPORTANT: Limit features to save memory!
    min_df=2,              # Ignore terms that appear in fewer than 2 documents
    max_df=0.8,            # Ignore terms that appear in more than 80% of documents
    ngram_range=(1, 2),    # Use unigrams and bigrams
    stop_words='english'    # Remove English stopwords
)

# Fit on training data and transform both train and test
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print(f"TF-IDF matrix shape (train): {X_train_tfidf.shape}")
```

```
print(f"TF-IDF matrix shape (test): {X_test_tfidf.shape}")
print(f"Number of features: {len(tfidf.get_feature_names_out())}")
```

```
TF-IDF matrix shape (train): (4000, 5000)
TF-IDF matrix shape (test): (1000, 5000)
Number of features: 5000
```

```
# Explore the TF-IDF features (optional)
# YOUR CODE HERE

# Hint: Look at the most important features
feature_names = tfidf.get_feature_names_out()
print("Sample features:", feature_names[:20])
```

```
Sample features: ['able' 'able buy' 'able order' 'able purchase' 'abr' 'absolute'
'absolute favorite' 'absolutely' 'absolutely delicious' 'absolutely love'
'absolutely loves' 'absorb' 'acai' 'acceptable' 'according' 'account'
'accustomed' 'achieve' 'acid' 'acidic']
```

Preprocessing Summary

Document your preprocessing choices:

1. **Preprocessing steps applied:**
2. **TF-IDF parameters chosen:**
3. **Final feature count:**
4. **Rationale for choices:**

✓ Part 4: Model Training

Objectives

- Train at least **2 different classification models**
- Compare their performance
- Document training time and resource usage

Recommended Models

Fast and Effective (Recommended for beginners)

- **Logistic Regression:** Fast, interpretable, works well with TF-IDF
- **Multinomial Naive Bayes:** Specifically designed for text classification

More Advanced (Optional)

- **Random Forest:** Ensemble method, can capture complex patterns
- **Support Vector Machine (SVM):** Good for high-dimensional data
- **XGBoost:** Powerful gradient boosting (but slower)

Tips

- **Start with simple models:** LogisticRegression and MultinomialNB are excellent choices
- **Use default parameters first:** Then tune if needed
- **Monitor training time:** Document how long each model takes
- **Save your models:** Use `pickle` or `joblib` to save trained models
- **For Random Forest:** Use `n_estimators=100` and `max_depth=20` to limit resources

```
# Import model libraries
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
```

```
import time
from datetime import timedelta
```

```
print("✅ Model libraries imported!")
```

```
✅ Model libraries imported!
```

✓ Model 1: Logistic Regression

```
# Train Logistic Regression model
# YOUR CODE HERE
```

```
# Hint:
```

```
print("Training Logistic Regression...")
start_time = time.time()
```

```
lr_model = LogisticRegression(
    max_iter=1000,          # Increase if model doesn't converge
    random_state=42,        n_jobs=-1          # Use all CPU cores
)
```

```
lr_model.fit(X_train_tfidf, y_train)
```

```
training_time = time.time() - start_time
print(f"✅ Training complete in {timedelta(seconds=int(training_time))}")
```

```
Training Logistic Regression...
```

```
✅ Training complete in 0:00:03
```

```
# Make predictions with Logistic Regression
```

```
# YOUR CODE HERE
```

```
y_pred_lr = lr_model.predict(X_test_tfidf)
```

```
print(f"Predictions shape: {y_pred_lr.shape}")
```

```
Predictions shape: (1000,)
```

✓ Model 2: Multinomial Naive Bayes

```
# Train Naive Bayes model
```

```
# YOUR CODE HERE
```



```
# Hint:
print("Training Multinomial Naive Bayes...")
start_time = time.time()

nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)

training_time = time.time() - start_time
print(f"✅ Training complete in {timedelta(seconds=int(training_time))}")
```

```
Training Multinomial Naive Bayes...
✅ Training complete in 0:00:00
```

```
# Make predictions with Naive Bayes
# YOUR CODE HERE

y_pred_nb = nb_model.predict(X_test_tfidf)
```

✓ Model 3 (Optional): Additional Model

Train a third model if you'd like to explore further!

```
# Train your third model (optional)
# YOUR CODE HERE

# Example: Random Forest
print("Training Random Forest...")
start_time = time.time()

rf_model = RandomForestClassifier(
    n_estimators=100,      # Number of trees
    max_depth=20,         # Limit depth to save memory
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train_tfidf, y_train)

training_time = time.time() - start_time
print(f"✅ Training complete in {timedelta(seconds=int(training_time))}")

y_pred_rf = rf_model.predict(X_test_tfidf)
```

```
Training Random Forest...
✅ Training complete in 0:00:00
```

Model Training Summary

Document your models:

Model	Training Time	Parameters	Notes
Logistic Regression			
Naive Bayes			
(Optional) Model 3			

✓ Part 5: Model Evaluation

Objectives

- Evaluate all models using multiple metrics
- Compare model performance
- Analyze errors using confusion matrices
- Interpret results in business context

Metrics to Calculate

- ✓ **Accuracy:** Overall correctness (but can be misleading with imbalanced data)
 - ✓ **Precision:** Of all positive predictions, how many were correct?
 - ✓ **Recall:** Of all actual positives, how many did we find?
 - ✓ **F1-Score:** Harmonic mean of precision and recall
 - ✓ **Confusion Matrix:** Visualize true vs predicted labels
 - ✓ **Classification Report:** Detailed metrics per class
-

Tips

- **Don't rely on accuracy alone:** Especially with imbalanced data
- **Understand the business context:** Is false positive or false negative worse?
- **Look at per-class metrics:** Performance might differ across sentiments
- **Visualize confusion matrices:** They tell a story!

```
# Import evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

print("✓ Evaluation metrics imported!")
```

✓ Evaluation metrics imported!

✓ Evaluate Model 1: Logistic Regression

```
# Calculate metrics for Logistic Regression
# YOUR CODE HERE

# Hint:
print("=" * 50)
print("LOGISTIC REGRESSION RESULTS")
```

```

print("=" * 50)

accuracy = accuracy_score(y_test, y_pred_lr)
precision = precision_score(y_test, y_pred_lr, average='weighted')
recall = recall_score(y_test, y_pred_lr, average='weighted')
f1 = f1_score(y_test, y_pred_lr, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print("\n")

print("Classification Report:")
print(classification_report(y_test, y_pred_lr))

```

```

=====
LOGISTIC REGRESSION RESULTS
=====
Accuracy: 0.8150
Precision: 0.7613
Recall: 0.8150
F1-Score: 0.7570

```

```

Classification Report:

```

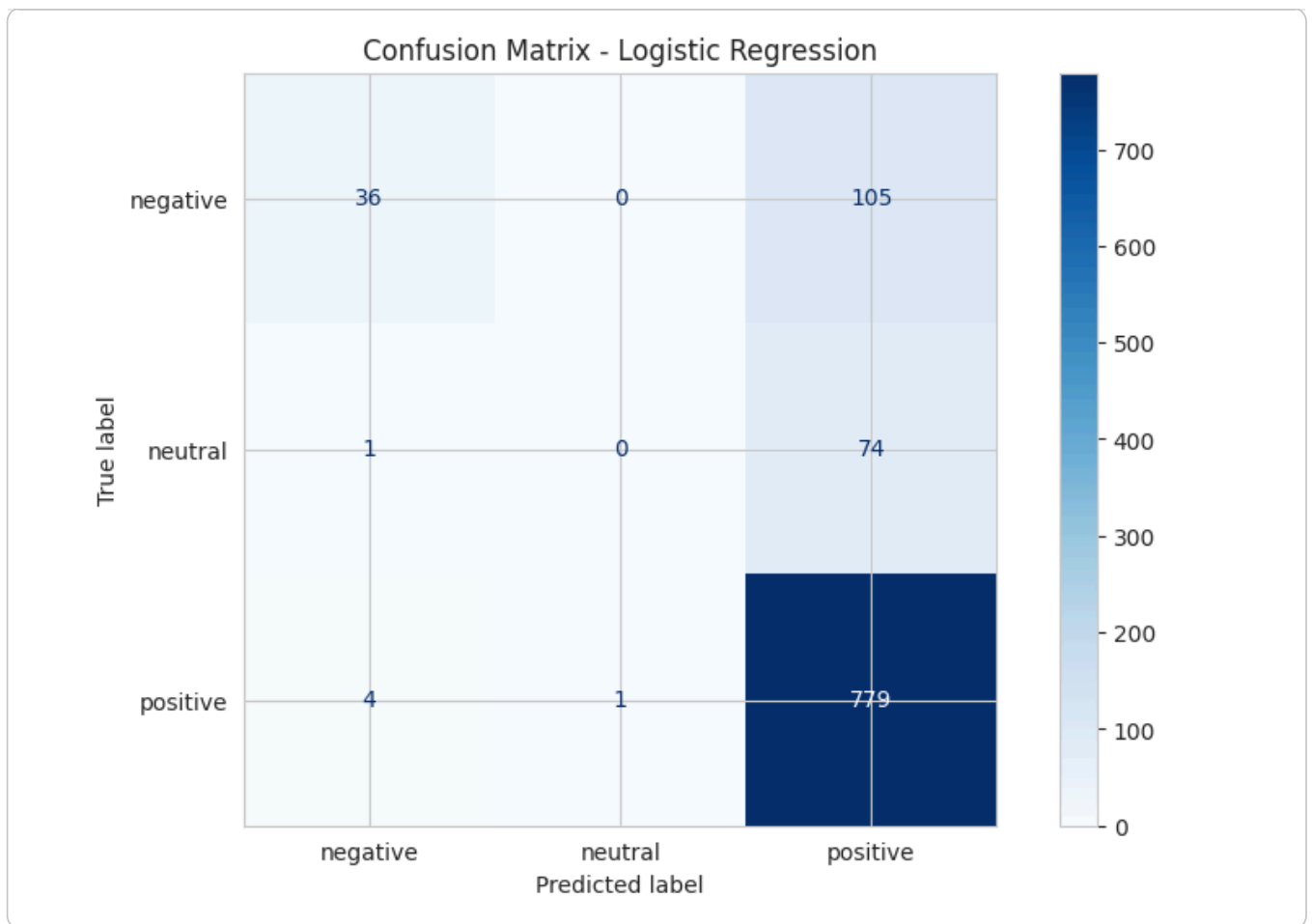
	precision	recall	f1-score	support
negative	0.88	0.26	0.40	141
neutral	0.00	0.00	0.00	75
positive	0.81	0.99	0.89	784
accuracy			0.81	1000
macro avg	0.56	0.42	0.43	1000
weighted avg	0.76	0.81	0.76	1000

```

# Plot confusion matrix for Logistic Regression
# YOUR CODE HERE

# Hint:
cm = confusion_matrix(y_test, y_pred_lr)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=lr_model.classes_)
disp.plot(cmap='Blues', values_format='d')
plt.title('Confusion Matrix - Logistic Regression')
plt.show()

```



Interpretation: Logistic Regression

Analyze the results:

1. **Overall performance:**
2. **Strengths:**
3. **Weaknesses:**
4. **Confusion matrix insights:**

✓ Evaluate Model 2: Naive Bayes

```
# Generate predictions for Naive Bayes
y_pred_nb = nb_model.predict(X_test_tfidf)

# Calculate metrics for Naive Bayes
print("=" * 50)
print("NAIVE BAYES RESULTS")
print("=" * 50)

accuracy = accuracy_score(y_test, y_pred_nb)
precision = precision_score(y_test, y_pred_nb, average='weighted')
recall = recall_score(y_test, y_pred_nb, average='weighted')
f1 = f1_score(y_test, y_pred_nb, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
```

```

print(f"Recall:    {recall:.4f}")
print(f"F1-Score:  {f1:.4f}")
print("\n")

print("Classification Report:")
print(classification_report(y_test, y_pred_nb))

```

```

=====
NAIVE BAYES RESULTS
=====

```

```

Accuracy:  0.7850
Precision: 0.7563
Recall:    0.7850
F1-Score:  0.6914

```

```

Classification Report:

```

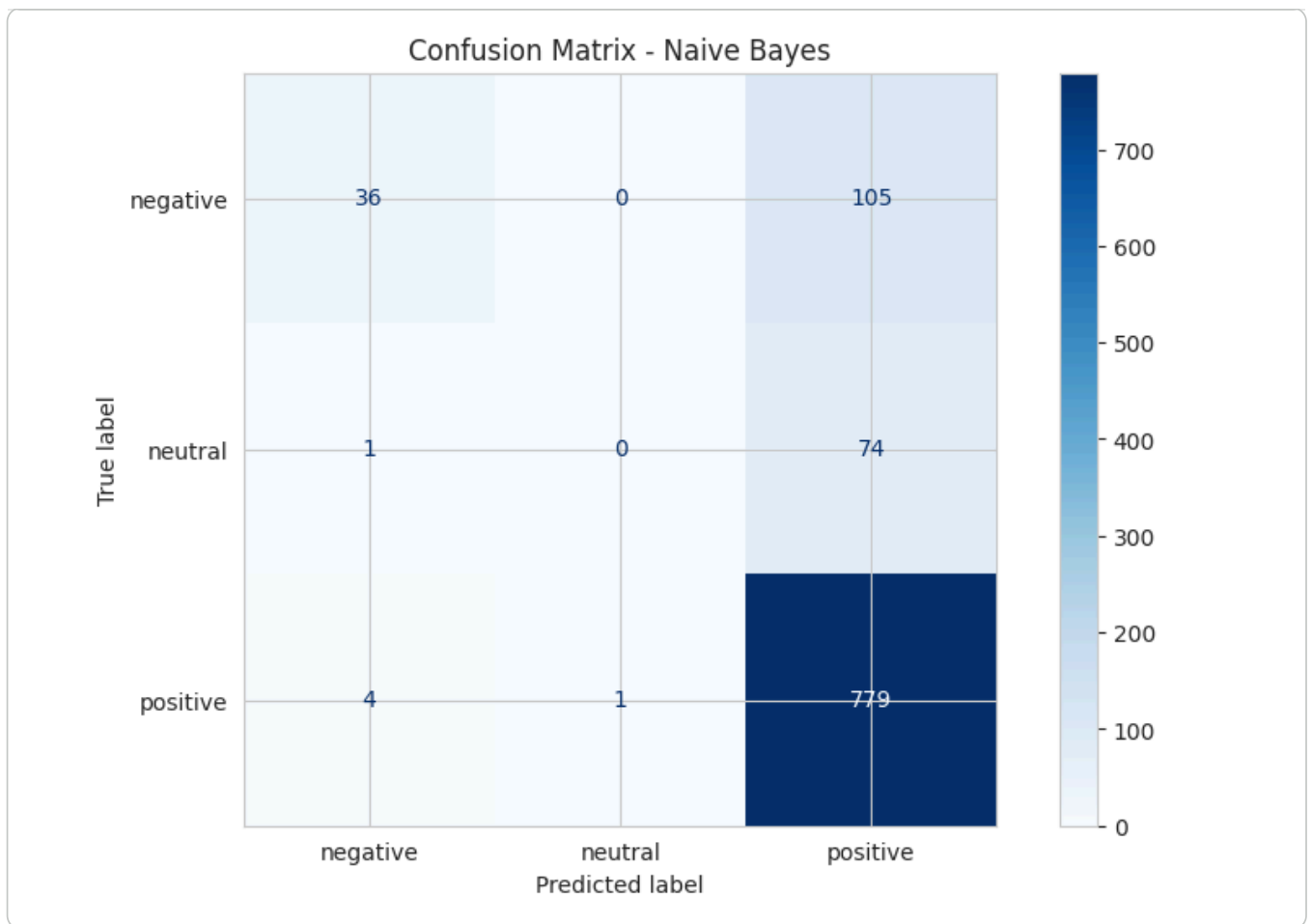
	precision	recall	f1-score	support
negative	1.00	0.01	0.01	141
neutral	0.00	0.00	0.00	75
positive	0.78	1.00	0.88	784
accuracy			0.79	1000
macro avg	0.59	0.34	0.30	1000
weighted avg	0.76	0.79	0.69	1000

```

# Plot confusion matrix for Naive Bayes
# YOUR CODE HERE

cm = confusion_matrix(y_test, y_pred_lr)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=lr_model.classes_)
disp.plot(cmap='Blues', values_format='d')
plt.title('Confusion Matrix - Naive Bayes')
plt.show()

```



Interpretation: Naive Bayes

Analyze the results:

1. **Overall performance:**
2. **Strengths:**
3. **Weaknesses:**
4. **Confusion matrix insights:**

✓ Evaluate Model 3 (Optional)

```
# Calculate metrics for your third model (if applicable)
# YOUR CODE HERE
```

```
# Plot confusion matrix for your third model (if applicable)
# YOUR CODE HERE
```

✓ Model Comparison

```
# Create a comparison table of all models

# Store metrics for Logistic Regression
acc_lr = accuracy_score(y_test, y_pred_nb)
```

```

prec_lr = precision = precision_score(y_test, y_pred_nb, average='weighted')
rec_lr = recall_score(y_test, y_pred_nb, average='weighted')
f1_lr = f1_score(y_test, y_pred_nb, average='weighted')

#Store metrics for Naive Bayes
acc_nb = accuracy = accuracy_score(y_test, y_pred_nb)
prec_nb = precision = precision_score(y_test, y_pred_nb, average='weighted')
rec_nb = recall_score(y_test, y_pred_nb, average='weighted')
f1_nb = f1_score(y_test, y_pred_nb, average='weighted')

# Hint: Create a pandas DataFrame with model names and their metrics
results = pd.DataFrame({
    'Model': ['Logistic Regression', 'Naive Bayes'],
    'Accuracy': [acc_lr, acc_nb],
    'Precision': [prec_lr, prec_nb],
    'Recall': [rec_lr, rec_nb],
    'F1-Score': [f1_lr, f1_nb]
})
# display(results)
print(results)

```

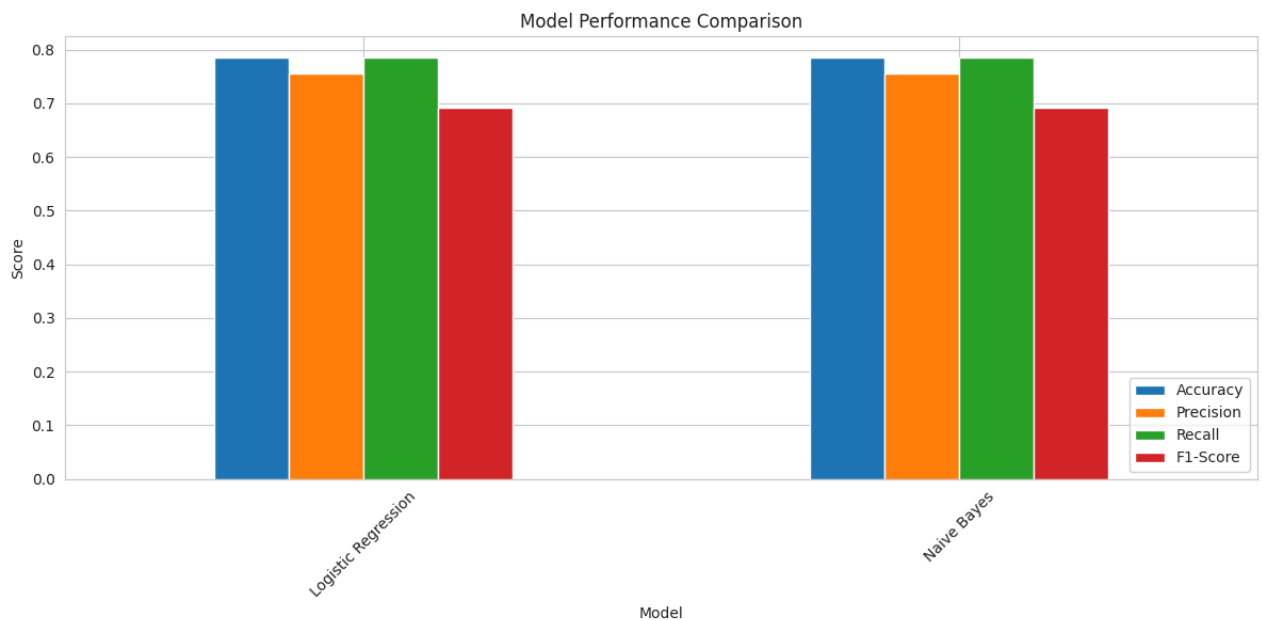
	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.785	0.756271	0.785	0.691449
1	Naive Bayes	0.785	0.756271	0.785	0.691449

```

# Visualize model comparison
# YOUR CODE HERE

# Hint: Create a bar plot comparing metrics across models
results.set_index('Model').plot(kind='bar', figsize=(12, 6))
plt.title('Model Performance Comparison')
plt.ylabel('Score')
plt.xticks(rotation=45)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

```



Model Comparison Analysis

Compare and contrast your models:

1. **Which model performed best?:**
2. **What metrics did you prioritize and why?:**
3. **Trade-offs between models:**
4. **Which model would you recommend for deployment?:**

✓ Error Analysis (Optional but Recommended)

```
# Analyze misclassified examples
# YOUR CODE HERE

# Hint: Find examples where the model was wrong
misclassified_indices = np.where(y_pred_lr != y_test)[0]
print(f"Number of misclassified examples: {len(misclassified_indices)}")

# Display some misclassified examples
for idx in misclassified_indices[:5]:
    print(f"\nText: {X_test.iloc[idx]}")
    print(f"True label: {y_test.iloc[idx]}")
    print(f"Predicted label: {y_pred_lr[idx]}")
    print("-" * 80)
```

Number of misclassified examples: 185

Text: not what i was expecting and will not order again i was looking for dried fruit this product
True label: negative
Predicted label: positive

Text: so i order my favorite jim beam seeds from amazon every couple of months i was running low s
True label: negative
Predicted label: positive

Text: one of my dogs had a bladder stone and needed surgery after her surgery the vet said to prev
True label: negative
Predicted label: positive

Text: the first ingredient in this formulation is mineral oil olive oil is the fourth i was underw
True label: negative
Predicted label: positive

Text: i absolutely love my coffee dark robust and full of deep flavor i have always liked starbuck
True label: negative
Predicted label: positive

Error Analysis Insights

What patterns do you notice in the errors?

1. **Common types of errors:**

2. **Why might these errors occur?:**
 3. **How could you improve the model?:**
-

✓ Part 6: Conclusion & Business Insights

Objectives

- Summarize your findings
 - Provide actionable business recommendations
 - Discuss limitations and future improvements
 - Reflect on what you learned
-



Executive Summary

Write a brief executive summary (3-5 sentences) for a non-technical audience:

[Your summary here]



Key Findings

List your main discoveries: